

Programming Fundamentals

- Overview
- Algorithms
- Flowcharts
- Pseudocode

- Computer programs

- ✓ a program is a set of instructions written in a specific programming language which a computer can execute to solve a given problem.

- ✓ sets of instructions that control a computer's processing of data

- ✓ **Program= Commands +instructions**

Programming

- A computer is not that intelligent. It cannot analyze a problem and come up with a solution on its own.
- A human must analyze the problem, develop the instructions for solving the problem, and then have the computer carry out the instructions. This person is referred to as **programmer, computer programmer, developer, coder, or software engineer**.
- A Programmer writes the **source code** for the program in one or more **programming languages**.

- How to communicate with a computer? We need to use a language that the computer can understand.
- A programming language is a formal language designed to communicate instructions to computers.
- Examples: C, C++, Java, PHP, and many more.



Programming Phases

- To write a program for a computer to follow, we must go through a two-phase process: **problem solving** and **implementation**.
- Before writing a program, we need to derive a procedure to solve the problem. To do this we use **algorithms**, **flowcharts**, **pseudocode**.

Problem-Solving Phase

1. Understand (define) the problem and what the solution must do.
 - What the computer program do?
 - What tasks will it perform?
 - What kind of data will it use, and where will get its data from?
 - What will be the output of the program?
 - How will the program interact with the computer user?
2. Derive a solution (Algorithm).
 - Specify the required data types and the logical sequences of steps that solve the problem.
3. Verify the solution.
 - Follow the steps exactly to see if the solution really does solve the problem.

Implementation Phase

1. Program (write) the Solution.

- Translate the algorithm (the general solution) into a programming language.

2. Test the program.

- Check the results. If you find errors, analyze the program and the algorithm to determine the source of the errors, and then make corrections.

- Overview
- **Algorithms**
- Flowcharts
- Pseudocode

Algorithms

- An algorithm is a sequence of steps required to solve a problem. A list of well-defined instructions to solve a problem.



Figure: al-Khuwarizmi

Algorithm Notations

- Algorithms can be expressed in many kinds of notation, including **natural languages, pseudocode, flowcharts, programming languages**.
 - Natural language expressions of algorithms tend to be verbose and ambiguous.
 - Pseudocode, flowcharts are structured ways to express algorithms that avoid many of the ambiguities common in natural language statements.
 - Programming languages are primarily intended for expressing algorithms in a form that can be executed by a computer.

Example:

Algorithm to Withdraw Money from ATM:

- Step 1: Insert your card into the ATM.
- Step 2: Choose the language you want.
- Step 3: Insert your PIN.
- Step 4: Insert the Amount.
- Step 5: Remove your card.
- Step 6: Take out the money & receipt.

Methods for developing an algorithm

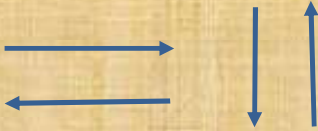
1. Define the problem: State the problem you are trying to solve in *clear* and *concise* terms.
2. List the *inputs* (information needed to solve the problem) and the *outputs* (what the algorithm will produce as a result)
3. Describe the steps needed to convert or manipulate the inputs to produce the outputs.
4. Test the algorithm: choose data sets and verify that your algorithm works!

- Overview
- Algorithms
- **Flowcharts**
- Pseudocode

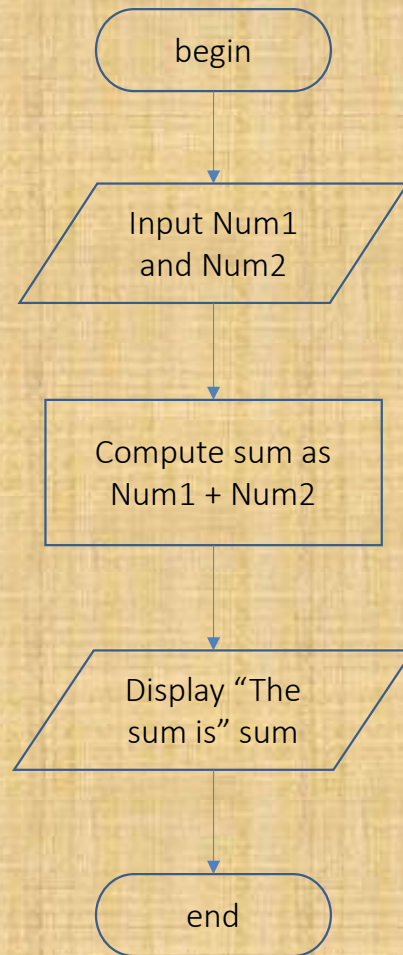
Flowcharts

- A flowchart is **graphical** representation of the steps to be followed to solve a problem. Consists of a set of **symbols** (notations) with a defined meaning to represent a specific activity.

Flowchart symbols

Symbol	Name/Meaning
	Terminal: Denotes the beginning or end of the algorithm/program.
	Input/Output: Denotes the input data or output data of the program
	Process: Denotes a process carried out. Ex. Calculation (addition, multiplication, etc)
	Decision: Denotes a decision (branch) to be taken.
	Flow lines: Denotes the direction of the logic flow.

Example: Add two numbers



- Overview
- Algorithms
- Flowcharts
- Pseudocode

Pseudocode

- The purpose of using pseudocode is that it is easier for people to understand than conventional programming language code
- Flowcharts can be thought of as a graphical alternative to pseudocode, but are more spacious on paper.
- Consists of natural language-like statements that precisely describe the steps of an algorithm or program.
- Statements describe *actions and* focuses on the *logic* of the algorithm or program.

Pseudocode Keywords

Keyword	Usage
begin .. end	Represents start & end of the algorithm
Input, get	Used to accept input
Display, print	Used to display output
Compute, assign, set, (=), ($\leftarrow$)	Used to assign value or variable to a variable
If .. Else .. EndIf Switch .. Case	Used to make decisions
While .. Do .. While For	Used to repeat statement(s)

Example: Add two numbers

Begin

input *number1, number2*

compute *sum* as $\text{number1} + \text{number2}$

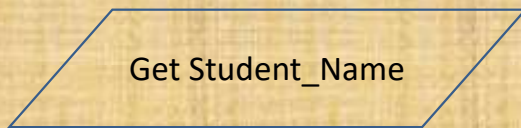
display 'The sum of the given two numbers is' *sum*

End

Six Basic Computer Operations

1. A computer can receive information

- **Flowchart**

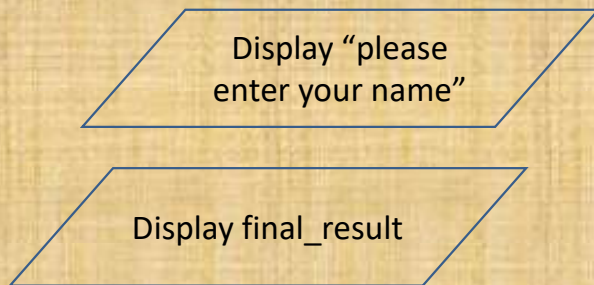


Pesudocode:

Input Student_Name

2. A computer can put out information

Flowchart



Pesudocode:

Print Student_Name

Display "Hellow There!"

* use any one of the words but use it consistently.

- **3. A computer can perform arithmetic**

+ for Add

- for Subtract

* for Multiply

/ for Divide () for Parentheses

- **Flowchart**

total_price = amount + vat

counter = counter + 1

Pesudocode:

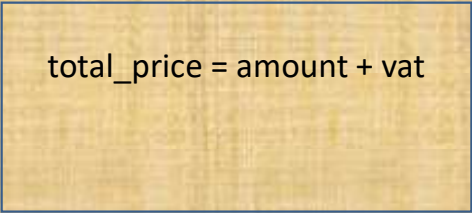
total_price = amount + vat

counter = counter + 1

* use any one of the words but use it consistently.

4. A computer can assign a value to a variable or memory location

Flowchart



```
graph TD; A[total_price = amount + vat];
```

total_price = amount + vat

Pesudocode:

Set counter 1
counter = 1

* use any one of the words but use it consistently.

5. A computer can compare two variables and select one of two alternate actions

- Single selection:

IF some condition Then

Do something when condition is true

- Double selection:

IF some condition Then

Do something when condition is true

Else

Do something else when condition is false

- Nested Selection: selection can be nested inside another selection.

IF some condition Then

IF another condition Then

Do something when both conditions are true

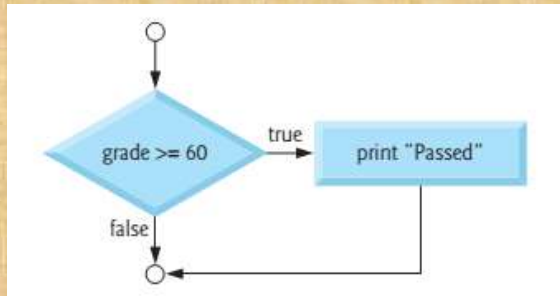
Else

Do something when the first condition is true and other condition is false

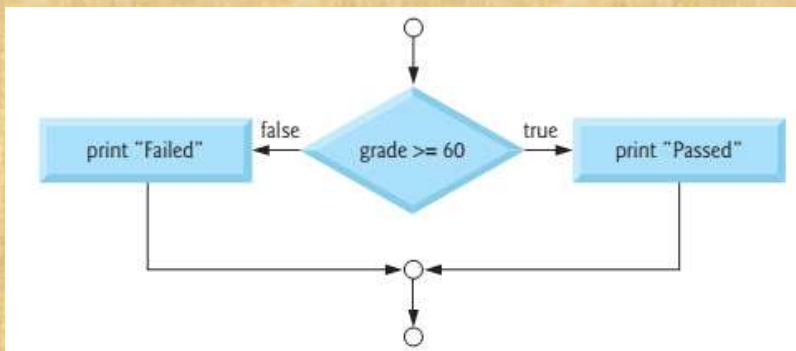
- **Selection**

- **Flowchart**

- **A) Single selection**



- **B) Double selection**



Pesudocode:

A) IF grade $\geq$ 60
 print "Passed"

B) IF grade $\geq$ 60
 print "Passed"
ELSE
 print "Failed"

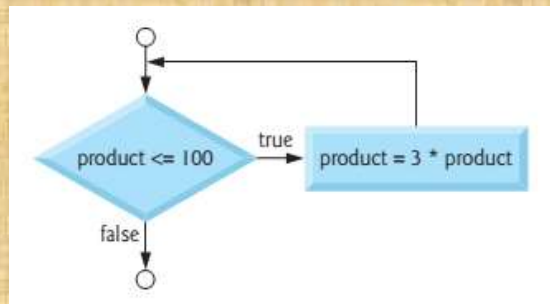
6. A computer can repeat a group of actions

- When there is a sequence of processing steps that need to be repeated while a condition is met.
- **While loop:** executes as long as the loop specified with the loop is true. When the condition becomes false the loop breaks. The condition test is at the top of the iterative statements.
- **Do .. While loop:** similar to while loop except that the condition test is at the end of the iterative statements. This means that the Do .. While loop will execute at least one time.
- **For loop:** consists of the following three parts:
 - ☐ Initialization expression ($i=0$)
 - ☐ Evaluation expression ($i<5$)
 - ☐ Increment/decrement expression ($i++$)

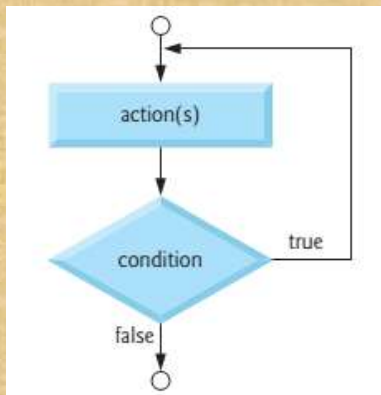
- Repetition

Flowchart

- A) While



- B) Do .. While

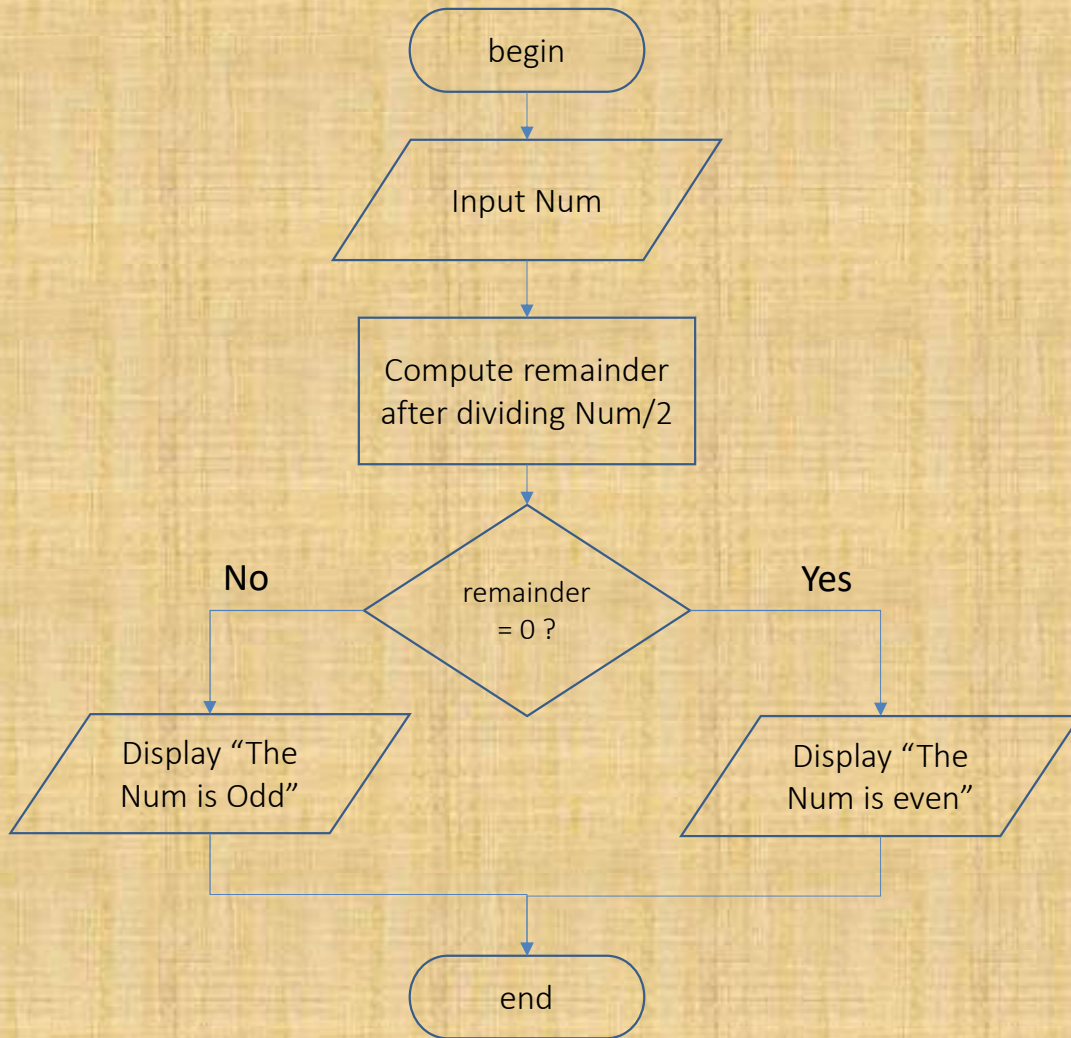


Pesudocode:

A) **While** product <= 100
 product = 3 * product

B) **Do**
 product = 3 * product
 While product <= 100

Example: Even/Odd number



Begin

input *number*

compute *remainder* after dividing number by 2

IF remainder equals 0

display 'The number is even'

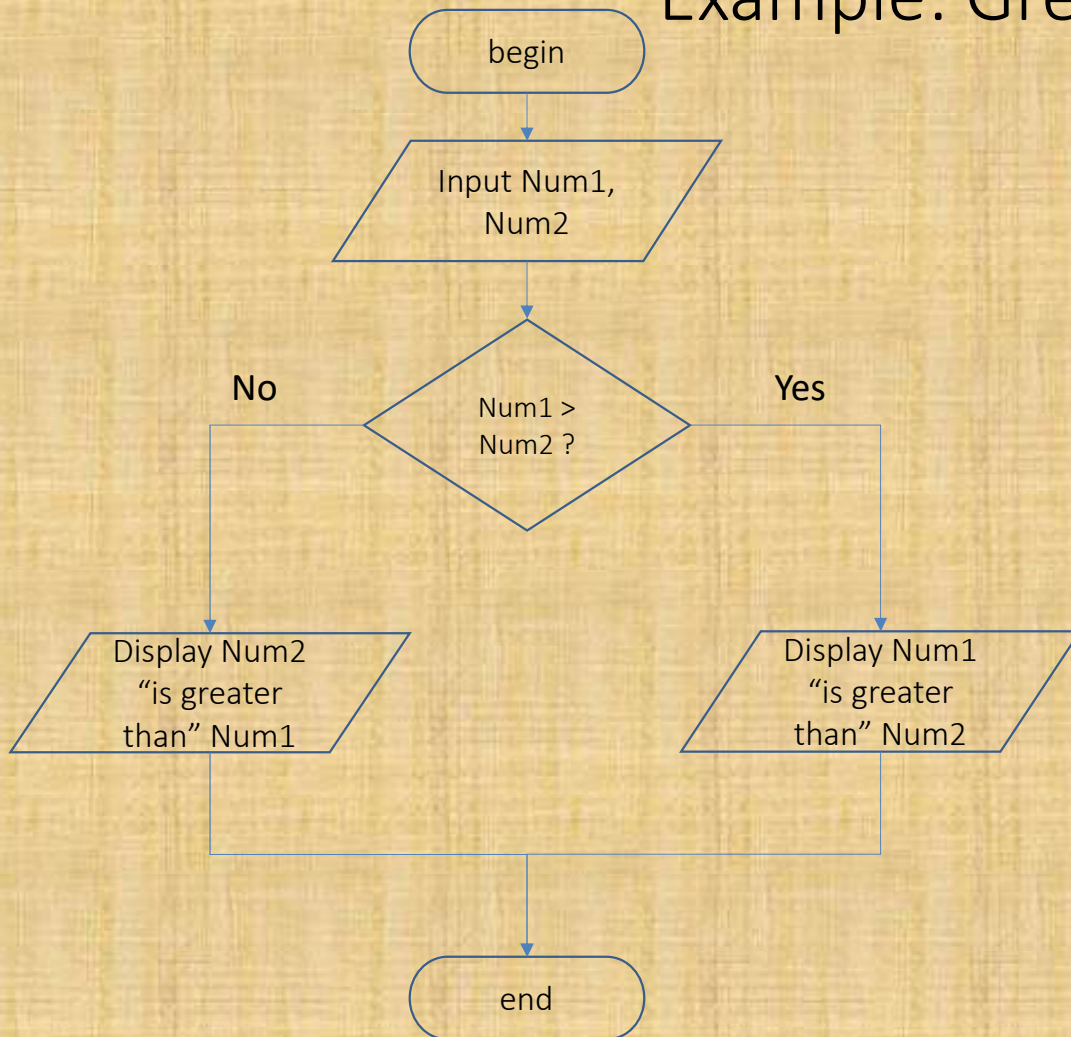
Else

display 'The number is Odd'

ENDIF

End

Example: Greater of Two Numbers



Begin

input *number1, number2*

IF *number1 > number2*

display *number1* 'is greater than' *number2*

Else

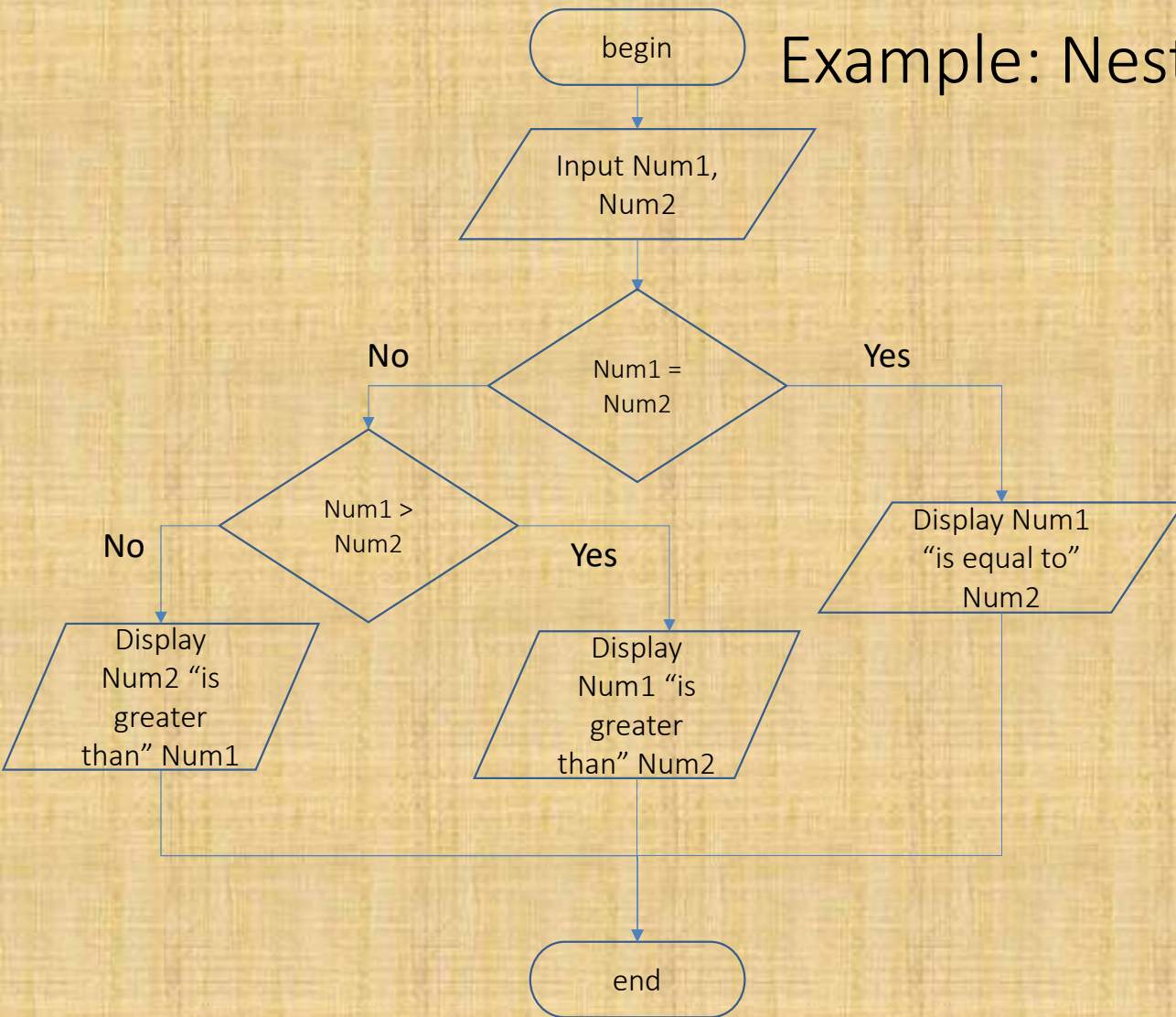
display *number2* 'is greater than' *number1*

ENDIF

End

Is this correct ?? What's wrong here??

Example: Nested Condition



Begin

input *number1, number2*

IF *number1 = number2*

display *number1 'is equal to' number2*

Else

IF *number1 > number2*

display *number1 'is greater than' number2*

Else

display *number2 'is greater than' number1*

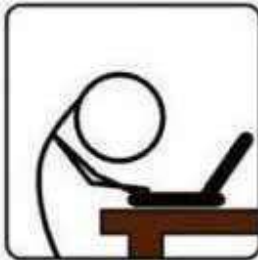
ENDIF

ENDIF

End

The Programmers Life

WORK



HOME



PLAY



SLEEP

